

Framebreak - Team20

Framebreak - Team20 (*student proposal*)

Assessment: ♦ Fit with adjustments · **Category:** Games · **Assessed:** 2026-09-21 · **Status:** awaiting instructor approval

Proposal (as submitted): You are a prisoner escaping with a magical camera: **snapping a real-world object materializes a functional 3D item inside your cell** to overcome obstacles. Camera power is limited, so every snapshot demands observation and spatial strategy — scout your surroundings, pick the best item, capture it to solve each area's escape puzzle. Named technical features: (1) **time-zone-synchronized procedural level generation**; (2) an **AI photo→3D pipeline** — a photo is classified to identify the object and assign gameplay attributes, sent to the backend where the background is stripped and an image-to-3D model generates a textured mesh, which a **C++ service reduces and fits with collision shapes** before it streams back and spawns, with a progress/status indicator; (3) **real-time 3D co-op multiplayer**; (4) an **attribute-matching puzzle system** that checks a spawned object's properties against a room's exit requirements (any item with the right attributes solves it). Loop/service: a C++ backend service turns the raw generated mesh into something the phone can use — mesh reduction, texture bake, collision generation; since every photo is unique nothing is reusable, so the service's throughput decides how many concurrent players are supported. Tools: a **desktop room editor** on the same C++ renderer (place geometry, set each room's exit requirement as an attribute list, set the camera charge limit); a server hosting the image-to-3D app (device sends photo → server returns a 3D model → used in the loop). Design features: photo-to-object manifestation; a **cooperative escape mode** (solo or online co-op, combine snapshots to solve complex puzzles); **dynamic time-of-day puzzles** (layouts change for morning/afternoon/evening/midnight); and a **hybrid monetization system** (base 10 snaps/area, a "large battery pack" raises it to 20; paid hints).

Why "with adjustments" (conditions for week-5 approval):

1. **Keep the load-bearing C++ on the team's side of the AI.** The module's performant-C++ requirement is met by *your mesh-processing service* (decimation, texture bake, collision-shape generation, streaming) + the game loop + the attribute-matching puzzle system + procedural generation — **not** by calling an external image-to-3D ML model. Treat the ML image-to-3D as one bounded component behind the service interface, and **build a fallback** (a curated library of pre-made meshes keyed by the classifier) so the game is fully demoable at the showcase even if the ML pipeline's latency or quality falls short. The classifier/image-to-3D itself is not a graded CS goal; the C++ service around it is.
2. **Rework the monetization.** Selling **extra snaps** (10 → 20 via the battery pack) and **paid hints** is pay-for-advantage, which breaks the module's clean-monetization rule (no pay-to-win, no manipulative loops). Make the snap budget a **balance/earned** decision and hints **free or earned**, and restrict any real-money purchases to **cosmetics**. (Design goal D8 in M2 owns this.)
3. **This is a genuine 3D project** (image-to-3D meshes, 3D items, 3D co-op) — the "3D (strong students only)" row applies, and it is the **central technical risk**. Scope the floor to single-player with a **curated object set** and a few rooms (the mesh service + attribute puzzle proven), with the ML photo→3D, real-time co-op and time-of-day generation as the stretch.

References:

- [Viewfinder on Steam](#) (a first-person puzzle game whose core verb is **turning photographs into real 3D geometry** you place into the world to solve puzzles, with a limited-film camera — the closest precedent for photo→3D-object manifestation under a snap budget)
- [Escape Simulator on Steam](#) (first-person escape-room puzzles playable **solo or in online co-op**, with a **room editor** and community rooms — the co-op-escape + room-editor precedent)
- [Scribblenauts — Wikipedia](#) (summon almost any object, whose **attributes** interact with the level to solve a puzzle — the "any item with the right attributes solves it" precedent)

Core Tech Requirements

Legend: ✓ Applies directly | ♦ Applies with domain adaptation | ● Optional | ✕ Not needed | » Second trimester

Core Tech Requirement	Framebreak
External Database	✓
Authentication »	✓
Analytics »	●
Real-time C++ Simulation Loop and/or C++ Performant Service	✓
Application Communication	✓
Continuous Integration	✓
Data-Driven Architecture	✓
Front-end + Back-end	✓
Custom Tools Development	✓
2D Graphical Output	✓
3D (strong students only)	✓
Networking System	✓

How each requirement applies:

Platform fit: **Main:** Native mobile (Native Android — the phone's camera captures the photo and the 3D game runs on-device; device: Pixel 9a) — the Native PC build is the development/tools vehicle (the desktop room editor on the same C++ renderer). **Extended:** a Linux server the team sets up by M2 (full use T2) hosting the image-to-3D app + the **C++ mesh-processing service** and accounts/store; Firebase optional for accounts/analytics.

Front end: A 3D renderer of the cell/rooms and the spawned meshes, the photo-capture/framing layer, the snap-budget + generation-status HUD, and

(co-op) the other player. **Languages:** Native — Kotlin Native Android UI + the C++ engine via JNI (Android Studio); OpenGL ES for the realtime 3D drawn in a GLSurfaceView; Web/desktop — SDL2/3 + OpenGL ES for the PC dev/room-editor build (same C++ renderer).

Back end: The client C++ loop (spawned-object physics, the attribute-matching puzzle check, procedural time-of-day layout) **and** the load-bearing **C++ mesh-processing service** on the server (mesh reduction, texture bake, collision-shape generation, streaming) sitting behind the image-to-3D ML component. **Languages:** C++ (loop + service).

Backend scope: Server-indicated (T2) — the image-to-3D + mesh service + accounts seam set up & basic-tested on the Linux host by M2, full use in T2 (the service's throughput sets the concurrent-player ceiling).

- *Loop and/or service:* two load-bearing pieces — the client fixed-timestep loop (spawned-object physics/collision, attribute-puzzle evaluation, time-of-day layout) **and** the C++ **performant service** that turns each unique generated mesh into a phone-ready asset (decimate → bake → collision → stream); its throughput decides concurrent players (measured in M1G3 / M2G1).
- *Data-driven:* object gameplay attributes (assigned at classification), room exit-requirements (attribute lists), the camera charge limit, and level/time-of-day config are all data, authored via the room editor — a new room or attribute is a data change.
- *2D:* the photo-capture/framing UI, the snap-budget counter and the generation-status indicator + HUD (custom-rendered) satisfy the 2D gate.
3D: ✓ **central** — the spawned meshes, the cell/room geometry and the co-op scene are real 3D (strong-students; the key risk). *Networking:* real-time 3D co-op (authoritative state, sync spawned objects across players).
- *Custom tool:* the **desktop room editor** on the same C++ renderer — place geometry, set each room's exit-requirement attribute list, set the camera charge limit.
- *External DB:* accounts, progression and the (reworked, cosmetic-only) store.
Auth »: accounts for co-op + store. *Analytics »:* optional — which objects players snap, where they get stuck.

Suggested Tech Goals

G1–G3 ★ are the common goals (fixed for all categories); the other 3 are picked from the Games pool in `milestone-goals-pool-per-category.pdf` (G1–G3 ★ common, G4+ picks).

M1:

- **G1 ★ Application Shell, Real-time C++ Simulation Loop, Input & Core Logic** — includes the standing-up of the **C++ mesh-processing service** (the load-bearing performant service).

Tech & dependencies

- **Tech: Web build:** SDL2/3 window + a fixed-timestep loop (C++17/20) for the escape gameplay, plus the first cut of the C++ service that decimates a mesh, bakes its texture and generates collision shapes; CMake, Git. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 3D is needed — no SDL on native, some screens are pure Native UI.
- **Prerequisites:** None — foundation of this project.
- **Feeds into:** Every other goal; the spawned-object physics and the attribute puzzle run on its tick; the service feeds M1G5.

- **G2 ★ Data-Driven Architecture of Core Objects / Entity System**

Tech & dependencies

- **Tech:** Entity/component model in C++ with object gameplay attributes, room exit-requirements (attribute lists), the camera charge limit and level/time-of-day config in JSON, hot-reloadable — a new room or attribute rule is a data change.
- **Prerequisites:** G1 (loop & entity hooks).
- **Feeds into:** M2G2 (the room editor edits this data), M2G13 (attribute-matching puzzle), M2G14 (time-of-day procgen).

- **G3 ★ Debugging, Profiling, Stress Test & CI** — includes profiling the mesh service's **throughput** (how many players it can support).

Tech & dependencies

- **Tech:** ImGui overlays (service queue, generation status, spawned-object state), Tracy profiling of the decimate→bake→collision pipeline for per-mesh latency + throughput, Catch2 tests for the attribute-puzzle

check and data loading, on a GitHub Actions Windows+Android matrix.

- **Prerequisites:** G1 (loop + service to instrument).
- **Feeds into:** M2G3 (CI hardening); the concurrent-player ceiling M2G1 validates.

- **G4 2D Graphics** (adapted to a **3D renderer**) — the cell/room geometry and the spawned meshes are the whole read of the game.

Tech & dependencies

- **Tech: Web build:** A 3D renderer (camera, lit meshes, texture/material for the streamed meshes, depth) plus the 2D HUD/photo-framing overlay; OpenGL ES, GLSL, stb_image, FreeType. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 3D is needed — no SDL on native, some screens are pure Native UI.
 - **Prerequisites:** G1 (loop & window).
 - **Feeds into:** M2G4 (advanced 3D shading/lighting for time-of-day); every visible frame; the room editor shares this renderer.
- **G5 Physics & Collision** — spawned objects interact physically (via the service's generated collision shapes) with the room and the puzzle.

Tech & dependencies

- **Tech:** C++ + GLM math, rigid-body + collision resolution using the collision shapes the service fits to each mesh, so a spawned item can be placed, stacked, or used to reach/block/trigger a puzzle element.
 - **Prerequisites:** G1 (tick + the service's collision shapes), G2 (object data).
 - **Feeds into:** The physical problem-solving; M2G13 (physics-based attribute checks).
- **G11 Networking Foundation** — the co-op multiplayer foundation (lobby, connection, state transport).

Tech & dependencies

- **Tech: Web build:** Sockets (Winsock2/BSD, or ENet/SDL_net) with a lobby/connection manager and a protocol that syncs room state and spawned objects between co-op players. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 3D is needed — no SDL on native, some screens are pure Native UI.
- **Prerequisites:** G1 (loop the co-op shares), G2 (state to serialize).

- **Feeds into:** M2G7 (match sync & prediction); the cooperative escape mode.

M2:

- **G1 ★ Architecture — Optimization & Platform Validation** — including the mesh service's concurrent-player throughput and the Linux image-to-3D host.

Tech & dependencies

- **Tech: Web build:** Tracy profiling of the client 3D loop and the service pipeline (per-mesh latency + concurrent throughput), ASan on MSVC, memory pools, and on-device validation (Pixel 9a); the Linux server (image-to-3D + mesh service + accounts) set up (account + software + basic connectivity test) by M2. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 3D is needed — no SDL on native, some screens are pure Native UI.
- **Prerequisites:** All M1 goals (hardens the M1 codebase).
- **Feeds into:** The Native-Android deliverable + a measured concurrent-player ceiling.
- **G2 ★ Content Editor — Core (Desktop Room Editor)** — place geometry, set each room's exit-requirement attribute list, set the camera charge limit, on the same C++ renderer as the game.

Tech & dependencies

- **Tech:** Dear ImGui + the shared C++ renderer: place/transform room geometry, author the exit-requirement attribute list, set the snap/charge limit, with round-trip JSON serialization, live preview and undo/redo.
- **Libraries/Software:** Dear ImGui, nlohmann/json, the project's OpenGL ES renderer (shared with the game).
- **Prerequisites:** G2 (room/attribute data model), M1G4 (renderer).
- **Feeds into:** All room content; the custom-tools showcase.

- **G3 ★ CI — CMake, Code Quality, Build Standards & Packaging**

Tech & dependencies

- **Tech: Web build:** CMake presets, clang-format + clang-tidy gates, reproducible Windows + Android Debug builds, tests (attribute puzzle, service pipeline, data loading) and packaging via GitHub Actions.
- Native build:** Android Studio (Kotlin Native Android UI + the C++

engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 3D is needed — no SDL on native, some screens are pure Native UI.

- **Prerequisites:** G3 (CI & test foundation).
- **Feeds into:** Release/showcase builds of every goal.

- **G7 Networking — Match Sync & Prediction** — the real-time 3D co-op: authoritative room state and synced spawned objects across players.

Tech & dependencies

- **Tech:** Authoritative state + snapshots, interpolation of the remote player, and consistent spawning/ownership of photo-generated objects across the co-op session.
- **Prerequisites:** M1G11 (networking foundation), M2G1 (optimization).
- **Feeds into:** The cooperative escape mode at showcase.

- **G13 Gameplay Systems II** — the **attribute-matching puzzle system** and the (reworked, non-pay-to-win) snap/charge economy + hints.

Tech & dependencies

- **Tech:** C++ systems checking a spawned object's attributes against a room's exit-requirement list (any qualifying item solves it), plus the snap-budget/charge economy and an earned/free hint system.
- **Prerequisites:** M1G2 (attributes/requirements), M1G5 (physical interactions).
- **Feeds into:** The core puzzle loop at showcase; T2 room expansion.

- **G14 Procedural Generation II** — the **time-of-day-synchronized** puzzle-layout variation (morning/afternoon/evening/midnight).

Tech & dependencies

- **Tech:** Rules that vary room layout / available solutions by the local time zone, with validation that each variant stays solvable given the room's exit requirements.
- **Prerequisites:** M1G2 (room data), M2G2 (rooms authored in the editor).
- **Feeds into:** The dynamic time-of-day replay variety at showcase.

Track Goals & Team Composition

Recommended composition: 6 CS + 2 Design + 1 Art (9) — because the interactive objects are **generated from photos rather than hand-authored**, the art surface is smaller than a typical 3D game (the cells/rooms, the time-of-day lighting, the camera/HUD and effects), so one artist can carry it; the **puzzle/attribute/room design is the larger surface** (attribute vocabulary, per-room exit-requirements, co-op puzzle design, the reworked economy, time-of-day variants), so two designers own it (rules: min 1 Design mandated, max 9 total, $D+A \leq 3$; see [team-formation.pdf](#)). If the environment art per time-of-day outgrows one artist, trade the 2nd designer for a 2nd artist. *Scope note (advisory):* the mesh-processing service + AI photo→3D + attribute puzzle + 3D co-op + time-of-day procgen + room editor is a very large, high-risk surface — the mesh service + attribute puzzle + a curated object set + a few rooms is the M1/M2 floor; the ML photo→3D, co-op and time-of-day generation are the stretch. **Prototype the mesh-service throughput and the curated-mesh fallback first.**

Track goals — suggested Design / UI-UX / Art / TEAM picks for this project; deliverables & quality ladders live in the track pool documents.

- **Design M1:**

- M1D1 ★ Features Design Document (the snap→materialize→attribute-solve escape loop, the co-op mode and time-of-day puzzles; primary benchmark e.g. Viewfinder, with a Singapore benchmark for the local-awareness criterion)
- M1D2 Core Gameplay Loop Prototype (a greybox proving snap → spawn → attribute-match → escape is fun with a limited snap budget) · prototype in Unity
- M1D4 Game Feel & Touch Interaction Specification (the photo-capture/framing feel and object placement, authored as tunable parameters)
- M1D5 Level & Progression Design (the escape rooms + the exit-requirement attribute design, authored to the room-editor data format)
- M1D6 Systems & Economy Design (the attribute vocabulary, the snap/charge budget and the — reworked, non-pay-to-win — economy)

- **Design M2:**

- M2D1 ★ Features Design Iteration & Showcase Readiness
- M2D3 Systems Balance & Tuning — Data-Driven (attribute/room/charge balance via the team's JSON, no code changes)
- M2D5 Playtesting & Iteration Program (test the puzzle clarity + the co-op flow, triage, re-test)
- M2D8 Monetization & Scope Integrity Design — the **required rework**: snaps/hints become balance/earned, real-money purchases restricted to cosmetics (clean-monetization, no pay-to-win) + a scope-integrity statement

- **UI/UX M1** (graded by the Design instructor for the Games category):

- M1U1 ★ UI Standards & Wireframe Baseline — readability standards for the photo-capture UI, the snap-budget counter and the generation-status indicator.
- M1U2 UX Flows & Wireframes — Menus → HUD — the scout → frame → snap → spawn → solve flow, solo and co-op.
- M1U3 Accessibility & Readability Foundations — legible status/generation feedback and reachable capture controls on a phone.

- **UI/UX M2:**

- M2U1 ★ Functional UI & Usability — capture, inventory, room and co-op UI navigable unaided on the Native-Android target.
- M2U2 ★ UX Validation & Final Polished UI — validated, polished capture + status UX tested with players on the Pixel 9a.
- M2U4 Onboarding & Tutorial Design — teach the snap→materialize→attribute-solve loop in the first area without a wall of text.

- **Art M1:** 1 artist (objects are photo-generated, so environment + UI are the art payload)

- M1A1 ★ Art Bible / Proof of Concept (the prison/escape visual direction — palette, time-of-day mood, how generated meshes sit in the scene)
- M1A3 Environment, Props & VFX Concepts (the cells/rooms, the snap/materialize VFX and the generation-in-progress effect)
- M1A4 UI Art Concepts & Typography (photo-capture UI, snap-budget counter, status indicator)

- **Art M2:**

- M2A1 ★ Critical Asset Production & In-Engine Visual Quality (rooms, time-of-day lighting, HUD implemented and consistent)
- M2A3 Environment Alive — Ambient Animation & Layering (the four time-of-day looks of the rooms)
- M2A4 UI Art — States & Feedback (framing / generating / spawned / out-of-charge states)

- **TEAM M1:**

- M1TA1 ★ Overall Audio Experience — initial BGM/SFX from the week-5 needs list present & working in the prototype (tense escape BGM, camera-shutter, materialize, puzzle-solve and error cues), suitable and correctly triggered.
- M1TA2 ★ Audio Engine Functionality — data-driven audio pipeline (add/replace files without recompiling) + reliable multi-channel playback.

- **TEAM M2:**

- M2TA1 ★ Overall Audio Experience — standard Games floor (escape BGM + shutter, materialize, place, solve, co-op and error SFX, ≥6 types), with time-of-day ambience.
- M2TA2 ★ Audio Engine Functionality — audio properties de/serialised and the full playback functionality the build needs (multi-channel mixing; a distinct materialize/solve cue).
- M2TP1 ★ Final Presentation — 7 min in front of class + instructors: live custom-tools showcase (the desktop room editor), prototype demo, team post-mortem (wk14)

*Generated by the Project Proposal Assessor skill · grounding: live folder
(Project 2 Technical adjustments/) · references verified 2026-09-21.*